VereadorCRM

Product Requirements Document

CRM inteligente para vereadores com atendimento automático via WhatsApp

Campo	Valor
Versão	1.0
Data	2025
Status	Pronto para desenvolvimento
Stack alvo	Node.js + React + PostgreSQL + Docker
Plataforma de desenvolvimento	Claude Code

1. Visão Geral do Produto

1.1 Problema

Vereadores municipais perdem demandas de munícipes diariamente. O fluxo hoje é caótico: mensagens chegam pelo WhatsApp pessoal do vereador ou de assessores, não são registradas, não têm acompanhamento e morrem sem resolução. Isso custa votos.

1.2 Solução

VereadorCRM é um SaaS multi-tenant que conecta um número de WhatsApp do gabinete a um motor de IA (Anthropic, OpenAI, Gemini ou endpoint customizado). A IA recebe mensagens de munícipes, responde de forma humanizada, classifica e registra cada demanda automaticamente em um CRM com dashboard. Assessores gerenciam a fila, atualizam status e o vereador acompanha relatórios com mapa político por bairro.

1.3 Proposta de valor por perfil

Perfil	Ganho principal
Vereador	Zero demanda perdida, mapa eleitoral atualizado em tempo real
Assessor	Fila organizada, histórico completo, resposta em segundos
Munícipe	Atendimento 24/7, resposta imediata, retorno garantido
Você (SaaS)	R$ 297–897/mês por cliente, 70%+ de margem bruta

2. Módulos do Sistema

O sistema é composto por 8 módulos. Cada módulo deve ser desenvolvido como uma unidade independente e testável. O Claude Code deve implementar na ordem listada abaixo.

#	Módulo	Prioridade	Descrição curta
1	Autenticação	MVP	Login, perfis e controle de acesso por role
2	WhatsApp + Webhook	MVP	Recepção de mensagens multi-provider
3	Motor de IA	MVP	Classificação e resposta automática
4	CRM — Demandas	MVP	Registro, fila, status e gestão
5	Configurações	MVP	IA, WhatsApp, vereador e sistema
6	Dashboard	MVP	Métricas e gráficos em tempo real
7	Mapa Político	V2	Demandas por bairro com insight eleitoral
8	Relatórios	V2	Exportação PDF, ranking, histórico

3. Módulo 1 — Autenticação e Controle de Acesso

3.1 Roles

Role	Acesso	Restrições
admin	Tudo	Único por tenant. Configura IA, WhatsApp, cria usuários
vereador	Dashboard, relatórios, demandas (leitura)	Não altera configurações técnicas
assessor	Demandas (leitura e escrita), munícipes	Não vê configurações nem relatórios financeiros

3.2 Funcionalidades obrigatórias

- Tela de login com email e senha — sem opção de cadastro público (admin cria usuários)
- Sessão via JWT com refresh token (access: 1h, refresh: 7d, armazenado em httpOnly cookie)
- Middleware de autenticação em todas as rotas da API
- Middleware de autorização por role em cada endpoint
- Tela de gerenciamento de usuários (só para admin): criar, editar role, desativar
- Trocar senha do próprio usuário (tela de perfil)
- Reset de senha por email com token de uso único válido por 1 hora
- Logout com invalidação do refresh token no servidor
- Bloqueio de conta após 5 tentativas de login — desbloqueio manual pelo admin

3.3 Schema do banco — tabela users

users: id (uuid PK), tenant_id (uuid FK), email (unique), password_hash, role (enum: admin|vereador|assessor), nome, ativo (bool), ultimo_login, tentativas_login, bloqueado_ate, criado_em, atualizado_em

3.4 Telas

- GET /login — formulário email + senha, link "Esqueci a senha"
- GET /perfil — dados do usuário logado, botão alterar senha
- GET /admin/usuarios — lista de usuários do tenant, botão criar, toggle ativo/inativo
- GET /admin/usuarios/novo — formulário: nome, email, role, senha temporária

4. Módulo 2 — Integração WhatsApp

4.1 Providers suportados

Provider	Tipo	Custo	Quando usar
Evolution API	Self-hosted (Docker)	Zero por mensagem	MVP, alto volume, controle total
Twilio	Cloud pago	R$ 0,08/mensagem	Confiabilidade máxima, suporte oficial
Z-API	Cloud BR	Plano fixo ~R$ 49/mês	Alternativa nacional simples

4.2 Fluxo de recepção de mensagem

1. WhatsApp do munícipe envia mensagem para o número do gabinete
2. Provider (Evolution/Twilio/Z-API) recebe e faz POST no webhook do backend
3. normalizer.js converte o payload para o formato interno unificado
4. messageProcessor.js recupera histórico da conversa (últimas 6 trocas)
5. aiRouter.js despacha para o provider de IA configurado
6. IA retorna resposta humanizada + JSON estruturado da demanda
7. store.js persiste a demanda no banco de dados
8. whatsappSender.js envia a resposta de volta ao munícipe

4.3 Configuração via painel (Módulo 5)

- Seletor do provider: Evolution | Twilio | Z-API
- Campos dinâmicos conforme provider selecionado:
  - Evolution: URL da instância, API Key, nome da instância
  - Twilio: Account SID, Auth Token, número de origem
  - Z-API: Instance ID, Token, Client Token
- Botão "Testar conexão" — envia mensagem de teste para número informado
- Status da conexão em tempo real (connected / disconnected / pending_qr)
- QR Code embutido no painel para conectar Evolution API sem sair do sistema
- Webhook URL gerada automaticamente para copiar e colar no provider

4.4 Endpoints de webhook

Método	Rota	Provider
POST	/webhook/evolution	Evolution API
POST	/webhook/twilio	Twilio WhatsApp
POST	/webhook/zapi	Z-API

5. Módulo 3 — Motor de IA

5.1 Providers de IA suportados

Provider	Modelos padrão	Custo estimado (500 msgs/mês)	Indicado para
Anthropic Claude	claude-haiku-4-5 / claude-sonnet-4-5 / claude-opus-4-6	R$ 2–15	Melhor compreensão de PT-BR
OpenAI	gpt-4o-mini / gpt-4o	R$ 1–8	Ampla adoção, Azure compatível
Google Gemini	gemini-2.0-flash / gemini-1.5-pro	R$ 0,50–4	Mais barato, alto volume
Custom / Ollama	Qualquer modelo OpenAI-compatible	Zero (local)	Testes, privacidade total

5.2 Editor de Prompt — texto livre com variáveis

O system prompt é editado diretamente como texto livre pelo admin no painel de configurações. O editor oferece syntax highlighting, inserção de variáveis dinâmicas por clique, templates prontos e simulação WhatsApp em tempo real com o prompt exato que será usado em produção.

5.3 Engine de variáveis dinâmicas

O prompt armazenado no banco contém placeholders no formato {{nome_variavel}}. A cada mensagem recebida, o messageProcessor.js resolve as variáveis antes de enviar para a IA. Variáveis disponíveis:

Variável	Valor injetado em runtime
{{vereador_nome}}	Nome completo do vereador (tenant_config.vereador_nome)
{{partido}}	Partido político (tenant_config.partido)
{{municipio}}	Município (tenant_config.municipio)
{{uf}}	Estado — sigla UF (tenant_config.uf)
{{legislatura}}	Período do mandato (tenant_config.legislatura)
{{horario_inicio}}	Início do expediente (tenant_config.horario_inicio)
{{horario_fim}}	Fim do expediente (tenant_config.horario_fim)
{{data_hoje}}	Data atual formatada em pt-BR
{{municipipe_nome}}	Primeiro nome do contato (municipipes.nome), se disponível
{{municipipe_phone}}	Telefone do munícipe formatado
{{bairros}}	Lista de bairros cadastrados (tenant_config.bairros)
{{orgaos_municipais}}	Secretarias e órgãos (tenant_config.orgaos_municipais)

Variável	Valor injetado em runtime
{{json_schema}}	Bloco completo do schema JSON de saída (ver seção 5.4)
{{regras_prioridade}}	Regras de classificação de prioridade (ver seção 5.4)

Variável não reconhecida no prompt (sem correspondente no banco) deve ser mantida literal — nunca lançar erro. Logar um warning no servidor para facilitar debug.

5.4 Schema JSON obrigatório de saída

Toda resposta da IA deve conter, após o texto ao munícipe, o JSON estruturado abaixo delimitado exatamente por |||JSON||| e |||FIM|||. O messageProcessor.js extrai esse bloco via regex e persiste como demanda no banco.

|||JSON||| { "categoria": "infraestrutura|saude|seguranca|social|educacao|outro", "subcategoria": "descrição curta do tipo de problema", "localizacao": "bairro ou endereço mencionado, ou nao informado", "prioridade": "alta|media|baixa", "resumo": "uma frase resumindo a demanda", "acao_sugerida": "o que o gabinete deve fazer", "precisa_retorno": true } |||FIM|||

Regras de prioridade que devem constar no prompt padrão:

- alta — risco à vida, saúde ou segurança pública
- media — infraestrutura degradada, serviços essenciais interrompidos
- baixa — melhorias, sugestões, elogios, esclarecimentos

5.5 Templates de prompt prontos

O editor oferece 4 templates como ponto de partida. Aplicar um template substitui o prompt atual após confirmação. Todos os templates já incluem variáveis dinâmicas e o bloco {{json_schema}}.

Template	Perfil	Característica principal
Padrão completo	Recomendado para a maioria	Tom empático, seções estruturadas, bairros e órgãos injetados
Gabinete formal	Câmaras de médio/grande porte	Linguagem institucional, respostas mais longas, tratamento formal
Foco eleitoral	Período pré-eleitoral	Reforça identidade do vereador, menciona projetos e realizações do mandato
Mínimo funcional	Testes e MVP rápido	Só o essencial: recebe, confirma e registra. Sem personalização adicional

5.6 Interface do editor no painel

A aba "Inteligência Artificial" nas Configurações deve implementar:

- Editor de texto com syntax highlighting em tempo real: seções (#, ##) em verde, variáveis {{...}} em laranja, delimitadores |||JSON||| em vermelho, comentários em cinza
- Numeração de linhas à esquerda do editor
- Sidebar esquerda com duas abas:
  - Aba Variáveis — lista todos os placeholders como chips clicáveis; ao clicar, insere no cursor atual do editor
  - Aba Templates — lista os 4 templates; ao clicar, confirma e substitui o conteúdo inteiro do editor
- Barra de ferramentas: botões Inserir seção, Inserir lista, Inserir JSON schema, Formatar (remove linhas em branco extras), Copiar (resolve variáveis antes de copiar)
- Contador em tempo real: caracteres, linhas, estimativa de tokens (chars / 3,8)
- Painel de preview direito com:
  - Badges das variáveis detectadas no prompt — vermelho se a variável não tiver valor cadastrado no tenant
  - Simulador WhatsApp: campo de mensagem de teste, chama a IA com o prompt exato (com API Key preenchida) ou simula localmente (sem API Key)
- Ctrl+S e botão Salvar persistem em tenant_config.ai_config.system_prompt
- Auto-save de rascunho em localStorage — recuperado se o usuário fechar sem salvar
- Botão Restaurar padrão carrega o template Padrão completo (com confirmação)

5.7 Histórico de conversa

- Manter até 20 turnos por número de telefone na tabela historico_conversa
- Usar as últimas 6 trocas como contexto em cada chamada à IA
- Limpar histórico após 24h de inatividade do número

5.8 Configuração técnica de IA

- Seletor de provider: Anthropic | OpenAI | Gemini | Custom
- Seletor de modelo — dropdown por provider ou texto livre para Custom
- Campo de API Key — type password, botão mostrar/ocultar, armazenado criptografado (AES-256)
- Campo Base URL — visível apenas para OpenAI e Custom
- Temperatura (slider 0.0–1.0, padrão 0.7) e máx. tokens de saída (slider 200–1200, padrão 800)
- Painel de custo estimado: tokens médios por mensagem × volume do mês × preço do modelo = R$/mês projetado

6. Módulo 4 — CRM de Demandas

6.1 Schema do banco — tabela demandas

demandas: id (uuid PK), tenant_id (uuid FK), from_phone (text), nome_contato (text), categoria (enum), subcategoria (text), localizacao (text), prioridade (enum: alta|media|baixa), resumo (text), mensagem_original (text), acao_sugerida (text), status (enum: nova|em_andamento|resolvida|cancelada), responsavel_id (uuid FK users), observacao (text), precisa_retorno (bool), protocolo (text gerado), criado_em (timestamptz), atualizado_em (timestamptz)

6.2 Funcionalidades da listagem

- Tabela com colunas: protocolo, munícipe, resumo, categoria, prioridade, localização, status, responsável, data
- Busca full-text por resumo, bairro, número de telefone e protocolo
- Filtros combinados: categoria, prioridade, status, responsável, período de datas
- Ordenação por qualquer coluna
- Paginação servidor-side (50 por página)
- Badge com contagem de novas demandas (atualiza a cada 30s via polling)
- Exportação da listagem filtrada para CSV

6.3 Funcionalidades do detalhe / edição

- Modal ou página de detalhe ao clicar em uma demanda
- Exibir: mensagem original, resposta enviada pela IA, histórico de conversa
- Editar status (nova → em_andamento → resolvida | cancelada)
- Atribuir responsável (dropdown dos assessores do tenant)
- Campo de observação interna (não visível ao munícipe)
- Botão "Responder via WhatsApp" — abre campo de texto e envia mensagem manual
- Histórico de alterações de status com data/hora e usuário
- Número de protocolo único gerado automaticamente (formato: MUN-2025-00001)

6.4 Schema — tabela municipipes

municipipes: id (uuid PK), tenant_id (uuid FK), phone (text unique por tenant), nome (text), primeiro_contato (timestamptz), ultimo_contato (timestamptz), total_demandas (int), tags (text[])

6.5 Módulo Munícipes

- Lista de todos os contatos com: nome (se informado), telefone, total de demandas, último contato
- Busca por telefone ou nome
- Ao clicar em um munícipe: ver todas as suas demandas, histórico de conversa completo

- Editar nome do contato manualmente
- Adicionar tags ao contato (ex: "liderança comunitária", "eleitor confirmado")

7. Módulo 5 — Configurações do Sistema

Página acessível apenas para o role admin. Organizada em abas. Todas as alterações são salvas por tenant_id no banco, sem afetar outros tenants.

7.1 Aba: Gabinete

- Nome do vereador
- Nome do município e UF
- Partido político
- Mandato (número da câmara, legislatura)
- Foto do vereador (upload, exibida no painel e no rodapé das mensagens automáticas)
- Saudação personalizada da IA (texto de boas-vindas ao munícipe)
- Horário de atendimento (exibido na mensagem automática fora do horário)
- Mensagem fora do horário — enviada automaticamente quando munícipe contata fora do horário definido

7.2 Aba: Integração WhatsApp

- Provider selecionado (Evolution | Twilio | Z-API) — altera campos exibidos
- Campos específicos do provider (ver seção 4.3)
- Status de conexão com indicador visual (verde / vermelho / amarelo)
- QR Code para Evolution API com botão "Atualizar QR"
- URL do webhook gerada automaticamente (botão copiar)
- Botão "Testar conexão" — envia "ping" para o número de teste
- Número de telefone de teste para validação

7.3 Aba: Inteligência Artificial

- Provider de IA (Anthropic | OpenAI | Gemini | Custom)
- Modelo — dropdown ou texto livre dependendo do provider
- API Key — campo password com botão mostrar/ocultar
- Base URL — visível para OpenAI e Custom
- System prompt — textarea com altura generosa, botão "Restaurar padrão"
- Temperatura da IA (slider 0.0 a 1.0, padrão 0.7)
- Máximo de tokens de saída (slider 200 a 1200, padrão 800)
- Botão "Testar IA" — campo de mensagem, envia e exibe resposta + JSON classificado
- Painel de custo estimado: tokens médios por mensagem, volume do mês, custo projetado em R$

7.4 Aba: Usuários e Acesso

- Lista de usuários do tenant com role, status e último login
- Criar novo usuário: nome, email, role, senha temporária (forçar troca no primeiro login)
- Editar role de usuário existente
- Ativar / desativar usuário

- Desbloquear conta bloqueada por tentativas excessivas

7.5 Aba: Notificações

- Email de notificação para novas demandas de alta prioridade (campo de email + toggle)
- Frequência do resumo diário por email: diário | semanal | nunca
- Webhook externo opcional: URL para receber eventos de nova demanda em JSON

8. Módulo 6 — Dashboard

8.1 Cards de métricas (topo)

Card	Valor exibido	Delta
Total de demandas	Contagem geral do mês	vs. mês anterior
Novas (não atendidas)	Status = nova	Número absoluto
Em andamento	Status = em_andamento	Número absoluto
Alta prioridade	Prioridade = alta + status != resolvida	Requer atenção
Taxa de resolução	% resolvidas / total	vs. mês anterior
Munícipes atendidos	Contagem unique de phones	vs. mês anterior

8.2 Gráficos

- Barras: demandas por categoria (infraestrutura, saúde, social, segurança, educação, outro)
- Donut: distribuição por prioridade (alta / média / baixa)
- Donut: distribuição por status (nova / em andamento / resolvida)
- Linha: volume de demandas por dia (últimos 30 dias)
- Barra horizontal: top 10 bairros por volume de demanda

8.3 Comportamento

- Auto-refresh a cada 30 segundos (polling simples, sem WebSocket no MVP)
- Filtro de período: últimos 7 dias | 30 dias | 90 dias | personalizado
- Todos os dados filtrados pelo tenant_id do usuário logado

9. Módulo 7 — Mapa Político (V2)

Módulo de análise estratégica para o vereador. Cruza dados de demandas com localização geográfica para gerar insights eleitorais.

9.1 Visualização por bairro

- Ranking de bairros por volume de demandas (barra horizontal proporcional)
- Categoria dominante por bairro
- Filtro por categoria para ver quais bairros concentram determinado tipo de demanda
- Mapa de calor opcional (se tiver coordenadas — via Leaflet.js com tiles OpenStreetMap)

9.2 Insights eleitorais automáticos

- Bairro com mais demandas abertas: "Concentre visitas aqui"
- Categoria dominante: "X% das demandas são de infraestrutura — pauta para requerimentos"
- Demandas de saúde: "Alta comoção emocional — material para redes sociais"
- Munícipes sem retorno há mais de 7 dias: "Risco de insatisfação — priorizar"

10. Módulo 8 — Relatórios (V2)

10.1 Relatório de desempenho mensal

- Gerado automaticamente no fechamento do mês
- Conteúdo: total de demandas, taxa de resolução, tempo médio de resposta, top categorias, top bairros
- Exportação em PDF (via Puppeteer ou html-pdf-node)
- Envio automático por email para admin e vereador se configurado

10.2 Relatório de demandas

- Exportação da listagem filtrada para CSV e PDF
- Campos incluídos: protocolo, munícipe, telefone, resumo, categoria, prioridade, localização, status, responsável, data de abertura, data de resolução

10.3 Relatório eleitoral

- Top 10 bairros por engajamento (demandas + interações)
- Ranking de assessores por demandas resolvidas
- Demandas resolvidas vs. abertas por período

11. Arquitetura Técnica

11.1 Stack tecnológica

Camada	Tecnologia	Justificativa
Backend API	Node.js 20 + Express	ESM, rápido, ecossistema amplo
Frontend	React 18 + Vite + TailwindCSS	SPA, componentes reutilizáveis
Banco de dados	PostgreSQL 16	Relacional, robusto, suporte a JSON
ORM	Drizzle ORM	Type-safe, migrations, sem overhead
Cache / Sessões	Redis (opcional no MVP)	Histórico de conversa, rate limit
WhatsApp	Evolution API (Docker)	Zero custo, auto-hosted
Autenticação	JWT + bcrypt	Stateless, seguro, simples
Proxy / HTTPS	Caddy	HTTPS automático Let's Encrypt
Infra	Docker Compose	Dev e prod idênticos, fácil deploy
Email	Nodemailer + SMTP	Reset de senha, notificações

11.2 Estrutura de diretórios

```
vereador-crm/ ├── backend/ │   ├── src/ │   │   ├── routes/        (webhook, api, auth) │
│   ├── services/       (ai, whatsapp, demandas, auth) │   │   ├── providers/       (ai:
anthropic, openai, gemini | wpp: evolution, twilio, zapi) │   │   ├── middleware/      (auth,
roles, rateLimiter) │   │   ├── db/            (schema drizzle, migrations) │   │   └── index.js
├── frontend/ │   ├── src/ │   │   ├── pages/          (Login, Dashboard, Demandas,
Municipipes, Mapa, Config) │   │   ├── components/      (Sidebar, Header, Charts, Modal,
Table, Forms) │   │   ├── hooks/          (useAuth, useDemandas, useMetricas) │   │   └──
api/            (client axios com interceptor de token) ├── docker-compose.yml ├── Dockerfile
(backend) ├── Caddyfile └── .env.example
```

11.3 Multi-tenancy

- Cada vereador (cliente SaaS) é um tenant independente
- Todas as tabelas têm coluna tenant_id — row-level security implícita
- Middleware verifica tenant_id em cada request autenticado
- Configurações de IA e WhatsApp são por tenant (tabela tenant_config)
- Isolamento total: nenhum tenant acessa dados de outro

12. Schema Completo do Banco de Dados

12.1 Tabelas e relacionamentos

Tabela	Chaves	Propósito
tenants	id (PK)	Cada cliente SaaS (gabinete de um vereador)
users	id (PK), tenant_id (FK)	Vereador, assessores e admin do gabinete
tenant_config	id (PK), tenant_id (FK unique)	Config IA, WhatsApp, prompt, notificações
demandas	id (PK), tenant_id (FK), responsavel_id (FK users)	Demandas dos munícipes
municipipes	id (PK), tenant_id (FK)	Contatos únicos por número de telefone
historico_conversa	id (PK), municipipe_id (FK)	Turnos da conversa com a IA
demanda_historico	id (PK), demanda_id (FK), user_id (FK)	Log de alterações de status
password_resets	id (PK), user_id (FK)	Tokens de reset de senha (TTL 1h)
refresh_tokens	id (PK), user_id (FK)	JWT refresh tokens ativos

12.2 Tabela tenant_config (configurações por tenant)

tenant_config: id, tenant_id (unique FK), vereador_nome, municipio, uf, partido, saudacao_personalizada, horario_atendimento_inicio, horario_atendimento_fim, mensagem_fora_horario, foto_url, whatsapp_provider (enum), whatsapp_config (jsonb — armazena campos específicos do provider criptografados), ai_provider (enum), ai_config (jsonb — api_key, model, base_url, system_prompt, temperatura, max_tokens), notificacao_email, notificacao_frequencia (enum), webhook_externo_url

13. Especificação da API REST

13.1 Autenticação

Método	Rota	Role	Descrição
POST	/auth/login	Público	Login — retorna access + refresh token
POST	/auth/refresh	Público	Renova access token via refresh token
POST	/auth/logout	Autenticado	Invalida refresh token
POST	/auth/forgot-password	Público	Envia email de reset
POST	/auth/reset-password	Público	Redefine senha com token
GET	/auth/me	Autenticado	Dados do usuário logado
PATCH	/auth/me/password	Autenticado	Altera própria senha

13.2 Demandas

Método	Rota	Role	Descrição
GET	/api/demandas	assessor+	Lista com filtros e paginação
GET	/api/demandas/:id	assessor+	Detalhe completo + histórico
PATCH	/api/demandas/:id	assessor+	Atualiza status, responsável, observação
POST	/api/demandas/:id/mensagem	assessor+	Envia mensagem manual ao munícipe
GET	/api/demandas/export	vereador+	Exporta CSV filtrado

13.3 Configurações, métricas e usuários

Método	Rota	Role	Descrição
GET	/api/config	admin	Lê configurações do tenant
PUT	/api/config/gabinete	admin	Salva dados do gabinete
PUT	/api/config/whatsapp	admin	Salva config WhatsApp
PUT	/api/config/ia	admin	Salva config IA
POST	/api/config/ia/testar	admin	Testa chamada de IA
POST	/api/config/whatsapp/testar	admin	Testa conexão WhatsApp
GET	/api/config/whatsapp/qr	admin	QR Code da instância Evolution
GET	/api/metricas	assessor+	Resumo para dashboard
GET	/api/municipipes	assessor+	Lista de contatos
GET	/api/municipipes/:id	assessor+	Detalhe + demandas do munícipe
GET	/api/usuarios	admin	Lista usuários do tenant

Método	Rota	Role	Descrição
POST	/api/usuarios	admin	Cria novo usuário
PATCH	/api/usuarios/:id	admin	Edita role / ativo

14. Segurança e Requisitos Não-Funcionais

14.1 Segurança obrigatória

- Senhas hasheadas com bcrypt (rounds: 12)
- JWT access token com expiração de 1 hora
- Refresh token armazenado em httpOnly cookie (não acessível por JavaScript)
- Rate limiting no endpoint de login: máximo 10 tentativas por IP por minuto
- Bloqueio de conta após 5 falhas consecutivas
- API Keys de IA e WhatsApp armazenadas criptografadas no banco (AES-256-GCM)
- Validação de tenant_id em cada query — nunca confiar só no JWT
- Sanitização de inputs contra SQL Injection e XSS
- CORS configurado para aceitar apenas a origem do frontend
- Headers de segurança via Caddy (HSTS, X-Frame-Options, X-Content-Type-Options)
- Webhook com validação de assinatura HMAC (para Evolution e Twilio)

14.2 Performance

- Resposta ao munícipe em menos de 5 segundos (end-to-end: recepção → IA → envio)
- API responde em menos de 200ms para listagens paginadas
- Imagens (foto do vereador) redimensionadas e comprimidas no upload (máx. 500KB)
- Índices no banco: tenant_id, from_phone, status, prioridade, criado_em

14.3 Confiabilidade

- Retry automático na chamada de IA: até 3 tentativas com backoff exponencial
- Fila de mensagens pendentes para envio WhatsApp (em caso de falha temporária)
- Logs estruturados com pino (JSON) — todos os eventos críticos registrados
- Health check em /health para monitoramento pelo Docker e Caddy

15. Instruções para o Claude Code

> *Esta seção é específica para o agente de desenvolvimento. Siga na ordem indicada.*

15.1 Ordem de implementação

9. Inicializar repositório: estrutura de pastas backend/ e frontend/ conforme seção 11.2
10. Configurar Docker Compose (dev e prod) e variáveis de ambiente
11. Implementar schema do banco com Drizzle ORM e rodar migrations iniciais
12. Implementar Módulo 1 — Autenticação completa (backend + frontend)
13. Implementar Módulo 2 — Webhook WhatsApp (normalizer + sender para os 3 providers)
14. Implementar Módulo 3 — Motor de IA (router + providers anthropic/openai/gemini/custom)
15. Implementar Módulo 4 — CRM: API de demandas + tela de listagem + modal de edição
16. Implementar Módulo 5 — Tela de Configurações com as 5 abas
17. Implementar Módulo 6 — Dashboard com métricas e gráficos
18. Testes de integração do fluxo completo: mensagem WhatsApp → IA → demanda → dashboard
19. Módulo 7 (Mapa Político) e Módulo 8 (Relatórios) — V2

15.2 Regras de implementação

- Nunca hardcodar API keys — sempre via variáveis de ambiente
- Nunca expor tenant_id de outros tenants — validar em TODA query
- Usar transações do banco para operações que escrevem em múltiplas tabelas
- Tratar erros de API de IA com retry e fallback para mensagem de desculpas ao munícipe
- Logs em JSON estruturado em produção, pretty-print em desenvolvimento
- Todas as datas no banco em UTC — conversão para fuso horário apenas na exibição
- Validação de inputs no backend com zod (não confiar no frontend)

15.3 Testes mínimos obrigatórios por módulo

Módulo	Teste obrigatório
Autenticação	Login correto retorna JWT. Login errado 5x bloqueia conta. Refresh token funciona.
Webhook	Payload Evolution normalizado corretamente. Payload Twilio normalizado corretamente.
IA	System prompt injeta nome do vereador. JSON de demanda é extraído corretamente.
CRM	PATCH de status atualiza demanda e registra no historico_demanda.
Config	PUT em /api/config/ia persiste API key criptografada. GET descriptografa.
Dashboard	GET /api/metricas retorna apenas dados do tenant autenticado.

16. Critérios de Aceite do MVP

O sistema é considerado MVP quando todos os itens abaixo forem verificados com sucesso.

#	Critério	Como verificar
1	Login de assessor não acessa configurações	Tentar GET /api/config — retornar 403
2	Mensagem enviada ao WhatsApp gera demanda no CRM	Enviar msg, verificar no banco e no dashboard
3	IA responde em menos de 5 segundos	Medir tempo end-to-end com curl
4	Troca de provider de IA sem reiniciar	Mudar config no painel, enviar msg teste
5	Dois tenants isolados	Login com tenant A não vê demandas do tenant B
6	Status de demanda atualizado por assessor aparece no dashboard	PATCH + GET /api/metricas
7	QR Code exibido no painel conecta WhatsApp	Escanear e receber mensagem de teste
8	Reset de senha por email funciona	Solicitar reset, clicar no link, trocar senha
9	Exportação CSV funciona com filtros aplicados	Filtrar por categoria e exportar
10	Deploy via docker compose up sobe tudo	VPS limpa, rodar e acessar em HTTPS

Apêndice — Referências e Links Úteis

APIs e SDKs

- Anthropic SDK: https://github.com/anthropic-ai/anthropic-sdk-python
- OpenAI SDK: https://github.com/openai/openai-node
- Google Generative AI: https://github.com/google/generative-ai-js
- Evolution API: https://doc.evolution-api.com
- Twilio WhatsApp: https://www.twilio.com/docs/whatsapp/api

Stack

- Drizzle ORM: https://orm.drizzle.team
- Zod (validação): https://zod.dev
- Pino (logs): https://getpino.io
- Caddy: https://caddyserver.com/docs
- Chart.js: https://www.chartjs.org

Infraestrutura

- VPS recomendada: Hetzner CX22 (2 vCPU, 4GB RAM) — aprox. R$ 50/mês
- Banco gerenciado alternativo: Supabase (free tier até 500MB)
- Email transacional: Resend (free 3.000 emails/mês) ou SendGrid

VereadorCRM PRD v1.0 — Documento preparado para desenvolvimento com Claude Code